CS 10: Operating Systems Laboratory – Assessment Task 7

Investigating Process Scheduling

Name: Rodnee Glen A. Martin

Year & Section: 3 BSCS A

Simulator: YASMIN CPU-OS Simulator 7.5.50

Date Performed: October 23, 2026

All exercises, including the optional ones, were done in the simulator. Figures are screenshots and log lines are copied exactly from the OS Activity Log. Time is in CPU ticks (1 instruction per tick); one FORNEXT process needs 365 ticks. A time slot of 10 ticks was used for every round robin and priority run. The program FORNEXT used in all runs is a for-next loop from 1 to 40 (source in the Appendix).

1. Loading a Program and Creating a Process

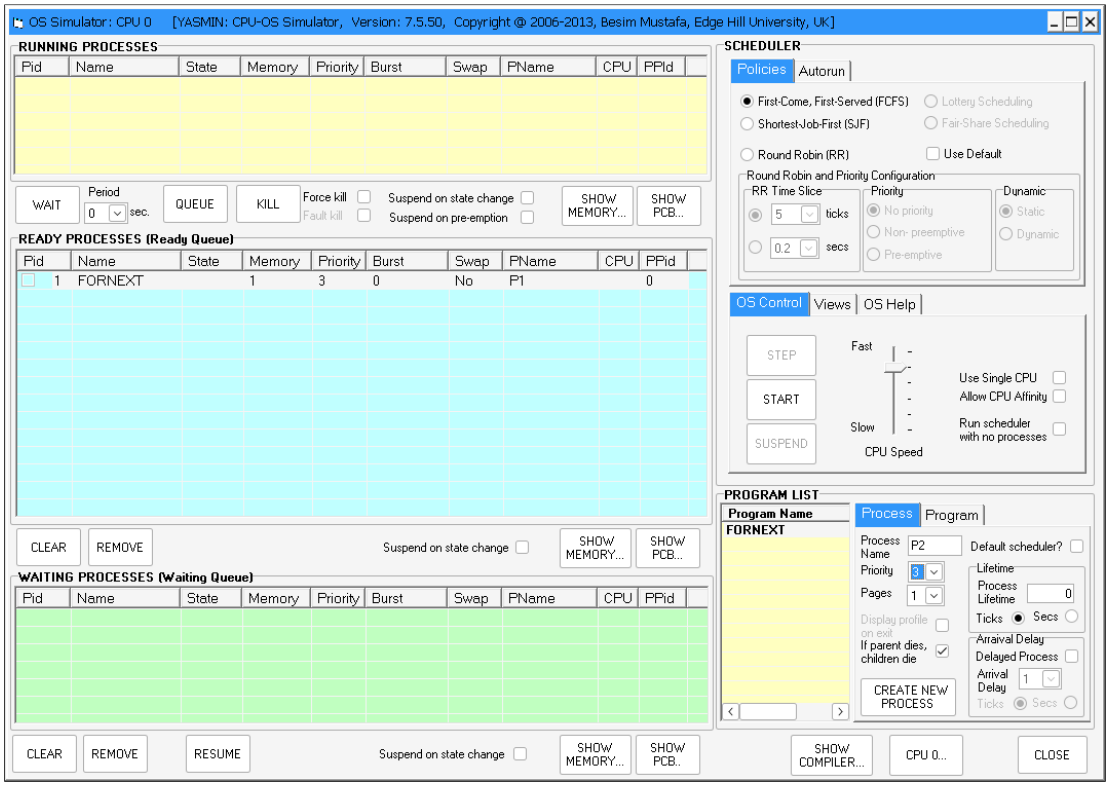


Figure 1. FORNEXT loaded in the Program List; one process (P1) created in the ready queue.

Can you explain the result of action 3 above?

Clicking CREATE NEW PROCESS created a process, an instance of the program FORNEXT, and put it in the READY queue. It got Pid 1, the name P1, priority 3, 1 page of memory, Burst 0 and PPid 0 (no parent). The log recorded “P1(1) loaded into the READY queue [Memory=1 pages, Priority=3]”. The process did not run yet because the scheduler was not started. The Process Name box changed to P2, so more processes of the same program can be created.

2. Running the Process

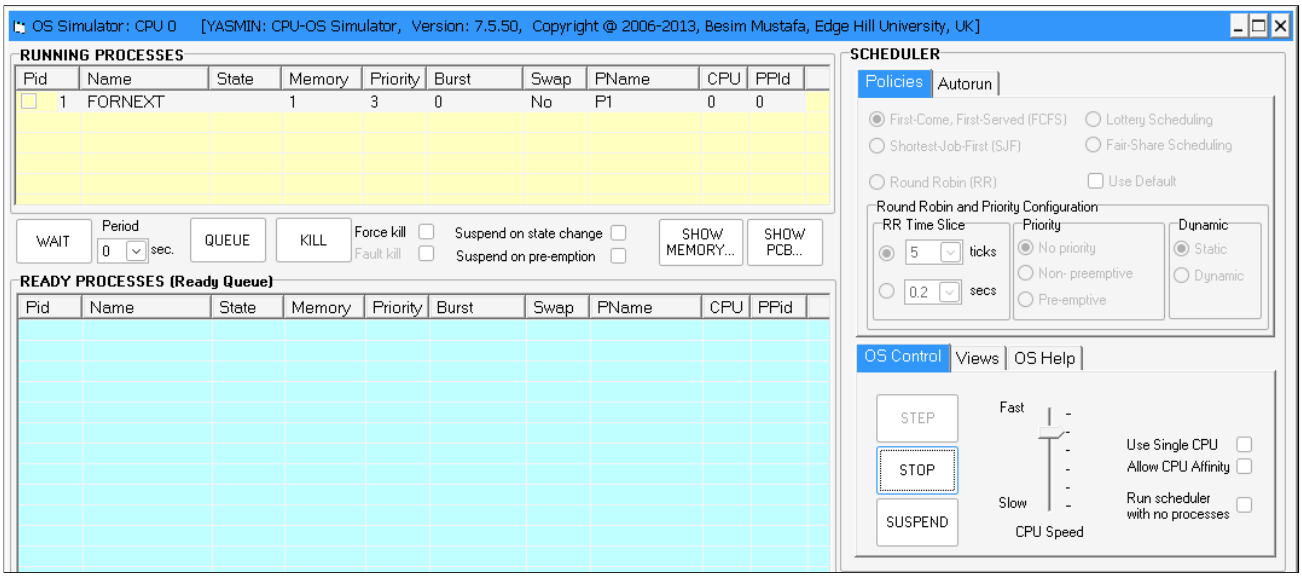


Figure 2. After START: P1 is in the Running Processes view on CPU 0.

Can you explain what happened?

START began a new scheduler session (First-Come-First-Served, First Fit memory). The scheduler moved P1 from READY to RUNNING on CPU 0, so it moved from the ready queue into the running view. START changed to STOP, SUSPEND became active and the policy options were greyed out. P1 ran all its instructions and terminated normally at tick 365 (about 16 seconds at the default speed), then it disappeared from the view.

What kind of information is displayed about a process?

Pid (process ID), Name (program name), State, Memory (pages used), Priority (lower number = higher priority), Burst (CPU burst in ticks), Swap (swapped out or not), PName (process name), CPU (CPU it runs on) and PPid (parent process ID). More details (registers, PC, waiting time) can be seen with SHOW PCB.

3. The Log

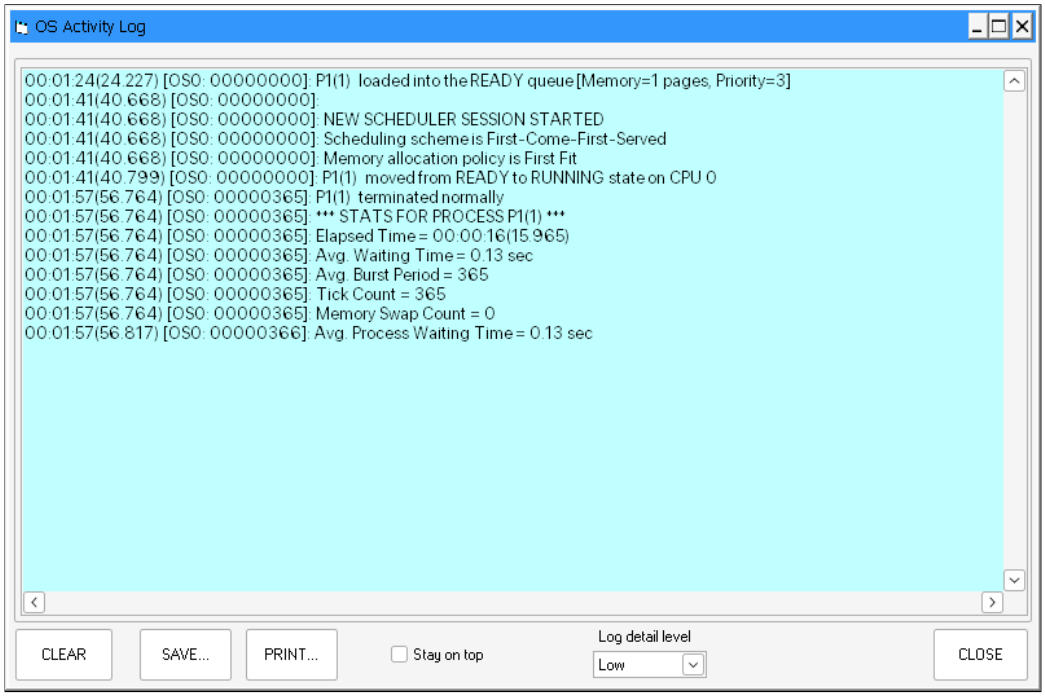


Figure 3. OS Activity Log after the first run.

What sort of information is displayed in the log window?

Every line has the time (hh:mm:ss) and the OS tick count [OS0: tick]. It shows when a process is loaded (memory and priority), the start of a scheduler session with the scheduling scheme and memory policy, every state change (READY to RUNNING on CPU 0, RUNNING to READY/WAITING), and how a process ended (terminated normally, killed or stopped). When a process ends, its statistics are shown: Elapsed Time, Avg. Waiting Time, Avg. Burst Period, Tick Count and Memory Swap Count, and at the end the Avg. Process Waiting Time of all processes. The log can be saved, printed or cleared.

4. Scheduling Policies

What is an algorithm?

An algorithm is a finite, ordered set of well-defined steps that solves a problem. A scheduling algorithm is the set of rules the OS uses to decide which ready process gets the CPU next and for how long.

4.1 First-Come-First-Served (steps 1–5)

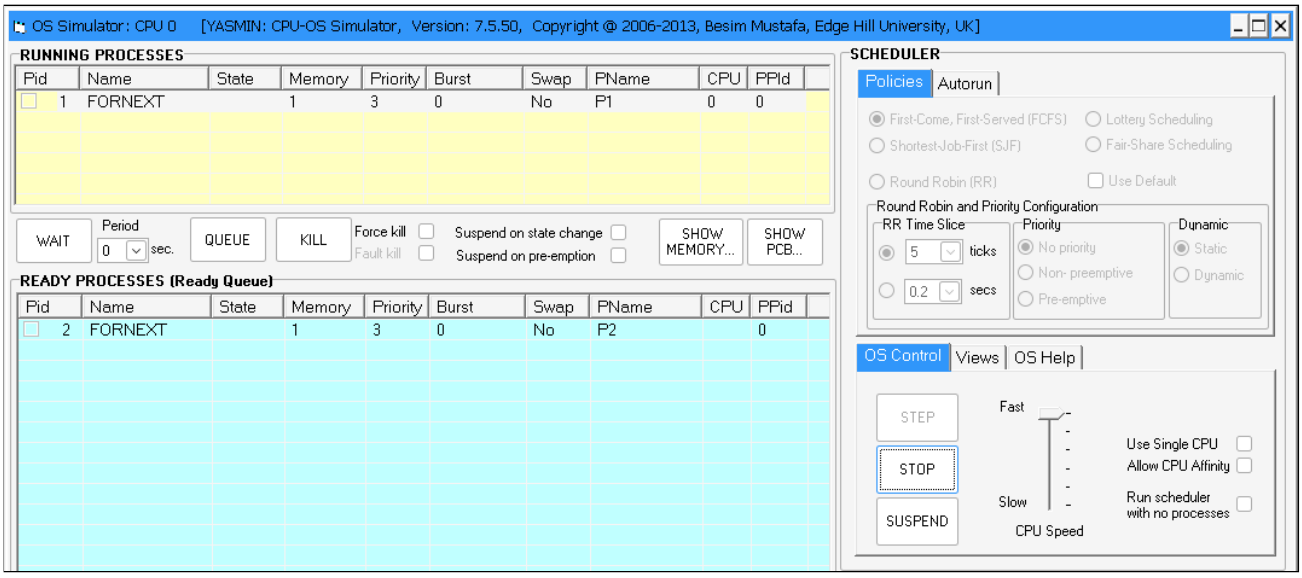


Figure 4. FCFS: P1 running while P2 waits in the ready queue.

```
00:02:36(35.821) [OS0: 00000000]: P1(1)  moved from READY to RUNNING state on CPU 0
00:02:41(41.174) [OS0: 00000365]: P1(1)  terminated normally
00:02:41(41.229) [OS0: 00000366]: P2(2)  moved from READY to RUNNING state on CPU 0
00:02:46(46.416) [OS0: 00000731]: P2(2)  terminated normally
00:02:46(46.484) [OS0: 00000732]: Avg. Process Waiting Time = 2.75 sec
```

Explain the behaviour of this scheduling policy.

Processes run in the order they arrived and a running process is never interrupted. P1 ran from tick 0 to 365 while P2 waited; only after P1 finished did P2 run, from 366 to 731. Each process had one burst of 365 ticks. P1 waited 0.05 s and P2 waited 5.45 s (average 2.75 s). FCFS is simple and has few context switches, but a long process makes all the others wait (convoy effect).

4.2 Priority, Non-pre-emptive (steps 6–13)

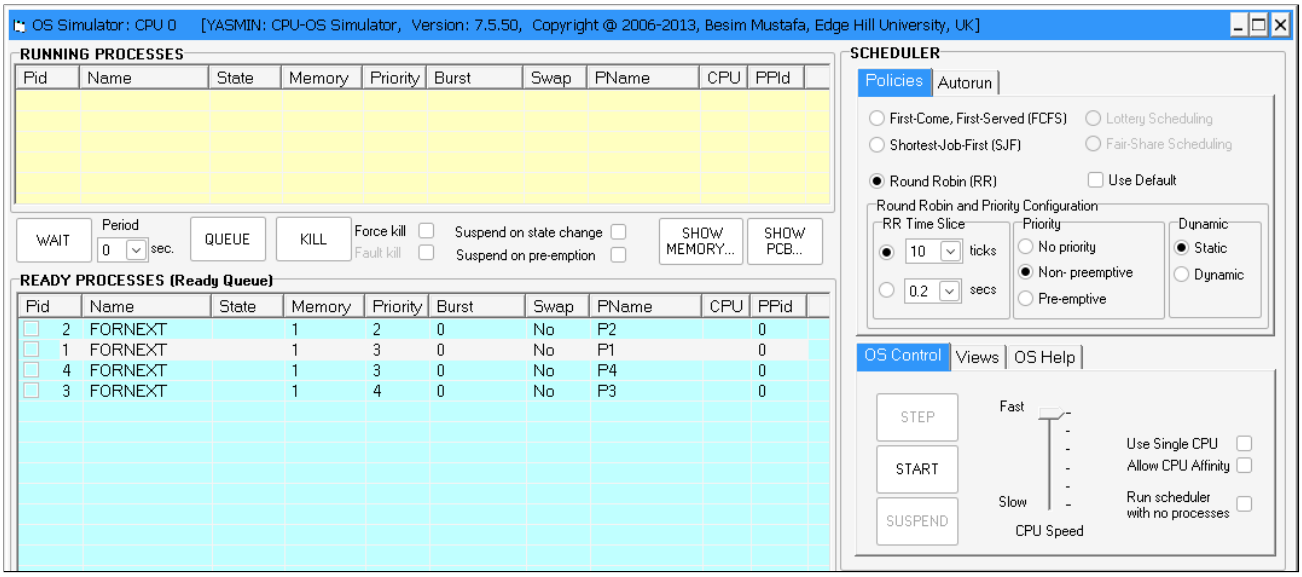


Figure 5. Ready queue sorted by priority: P2 (2), P1 (3), P4 (3), P3 (4).

```
Scheduling scheme is Round Robin using Static Priority (Non-preemptive): Time slots 10 ticks
[OS0: 00000000]: P2(2)  moved from READY to RUNNING state on CPU 0
[OS0: 00000365]: P2(2)  terminated normally
[OS0: 00000366]: P1(1)  moved from READY to RUNNING state on CPU 0
[OS0: 00000376]: P4(4)  moved from READY to RUNNING state on CPU 0   (P1 and P4 alternate)
[OS0: 00001091]: P1(1)  terminated normally
[OS0: 00001097]: P4(4)  terminated normally
[OS0: 00001098]: P3(3)  moved from READY to RUNNING state on CPU 0
[OS0: 00001463]: P3(3)  terminated normally
```

Explain the behaviour of this scheduling policy.

The process with the highest priority (smallest number) runs first. P2 (priority 2) ran first from tick 0 to 365; at the end of every 10-tick slot it was chosen again because nothing had a higher priority. Then P1 and P4, both priority 3, took turns every 10 ticks and finished at 1091 and 1097. P3 (priority 4) ran last, 1098 to 1463, and waited the longest (23.42 s against 0.03, 0.44 and 0.45 s). Non-pre-emptive means a newly arrived higher-priority process does not interrupt the running one; it waits until the slot ends or the process finishes. Low-priority processes can starve.

4.3 Round Robin (steps 14–21)

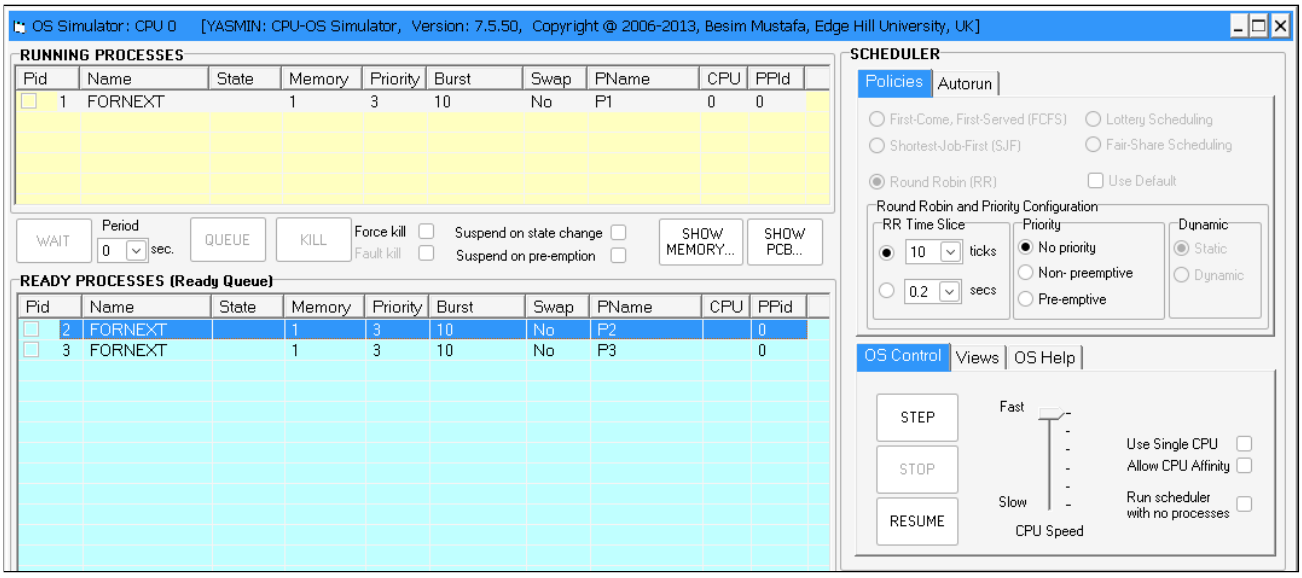


Figure 6. RR, 10 ticks, suspended: P1 running, P2 (selected) and P3 ready.

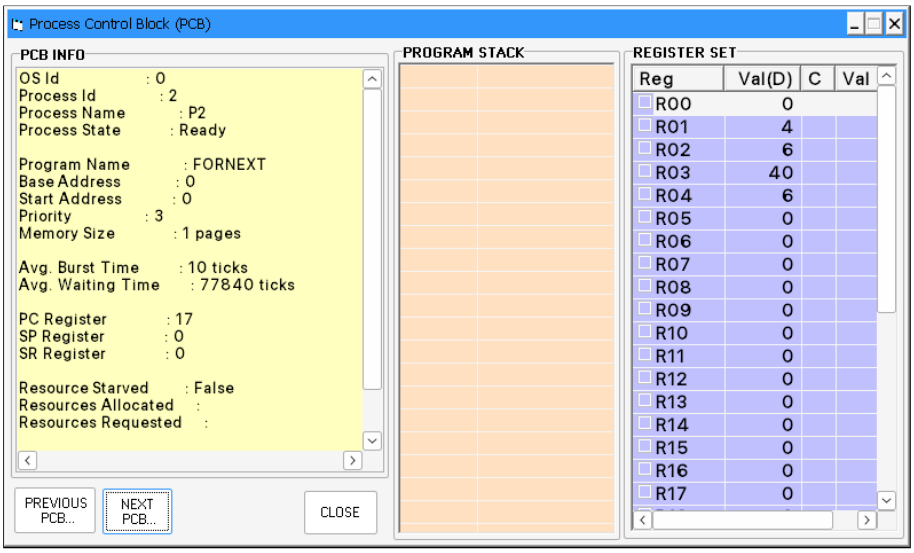


Figure 7. PCB of P2.

What do you see?

The Process Control Block (PCB) window of P2. It has the PCB INFO (IDs, state, program, addresses, priority, times and registers PC/SP/SR), the PROGRAM STACK (empty) and the REGISTER SET (R01 = 4, R02 = 6, R03 = 40, R04 = 6).

Observe the number against the PC Register. What is the significance of this number?

PC = 17 is the address of the next instruction P2 will execute when it gets the CPU again (CMP R03,R01, the loop test). The PC is saved in the PCB at every context switch, so the process continues exactly where it stopped instead of starting over. It matches the registers: P2 had run 30 instructions, so the loop counter is 4 (R01) and the sum is 6 (R02/R04).

What other important information do you see about this process?

Process Id 2, name P2, state Ready, program FORNEXT, base and start address 0, priority 3, memory 1 page, average burst 10 ticks, average waiting time, SP and SR registers (0), Resource Starved = False with no resources allocated, and the links to the previous PCB (1) and next PCB (3).

```
[OS0: 00000000]: P1(1)  moved from READY to RUNNING state on CPU 0
[OS0: 00000010]: P2(2)  moved from READY to RUNNING state on CPU 0
[OS0: 00000020]: P3(3)  moved from READY to RUNNING state on CPU 0
[OS0: 00000030]: P1(1)  moved from READY to RUNNING state on CPU 0   ...
[OS0: 00001085]: P1(1)  terminated normally
[OS0: 00001091]: P2(2)  terminated normally
[OS0: 00001097]: P3(3)  terminated normally
```

Explain the behaviour of this scheduling policy.

Each process gets the CPU for at most one time slot (10 ticks), then it is put at the back of the ready queue and the next one runs: P1, P2, P3, P1, ... Each process needed 37 turns, so they all finished close together (1085, 1091, 1097) with nearly equal waiting times (0.49, 0.5, 0.51 s). RR is fair and gives good response time, but it causes many context switches. If the slot is too long it behaves like FCFS; if it is too short the switching overhead is high.

4.4 Priority, Pre-emptive (steps 22–28)

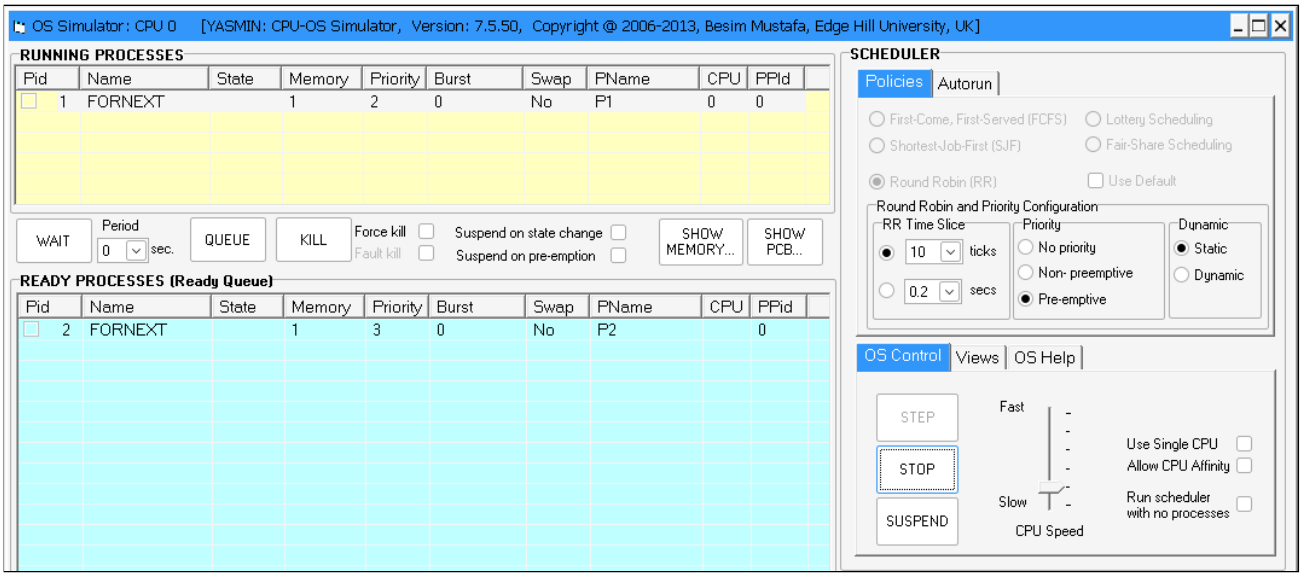


Figure 8. Before: P1 (priority 2) running, P2 (priority 3) ready.

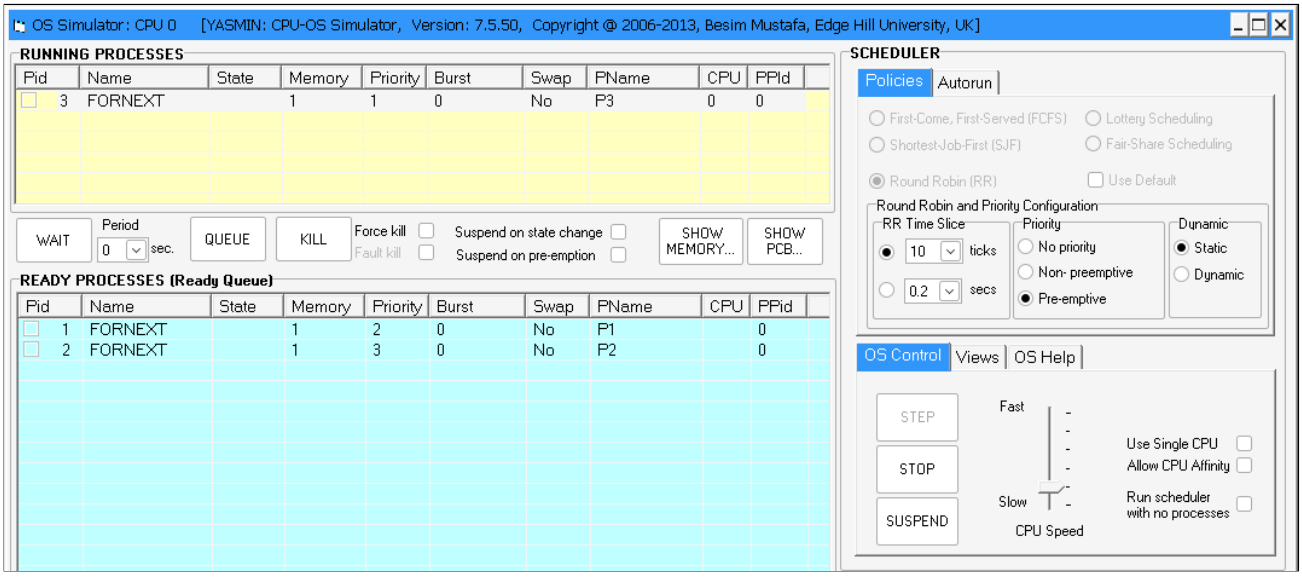


Figure 9. After creating P3 with priority 1: P3 running, P1 and P2 ready.

Which process is running?

P3 (priority 1, the highest).

Which processes are in the ready queue?

P1 (priority 2) followed by P2 (priority 3).

```
[OS0: 00000000]: P1(1)  moved from READY to RUNNING state on CPU 0
[OS0: 00000004]: P3(3)  loaded into the READY queue [Memory=1 pages, Priority=1]
[OS0: 00000004]: P1(1)  moved from RUNNING to READY state
[OS0: 00000004]: P3(3)  moved from READY to RUNNING state on CPU 0
[OS0: 00000369]: P3(3)  terminated normally
[OS0: 00000731]: P1(1)  terminated normally
[OS0: 00001097]: P2(2)  terminated normally
```

Explain the behaviour of this scheduling policy.

When P3 arrived at tick 4 with a higher priority, P1 was pre-empted immediately (in the middle of its slot) and P3 got the CPU. P3 ran until 369, then P1 continued (370 to 731) and P2, the lowest priority, ran last (732 to 1097) and waited the longest (41.51 s). Pre-emptive priority gives the fastest response to important processes, but low-priority processes can starve.

4.5 WAIT (steps 29–35)

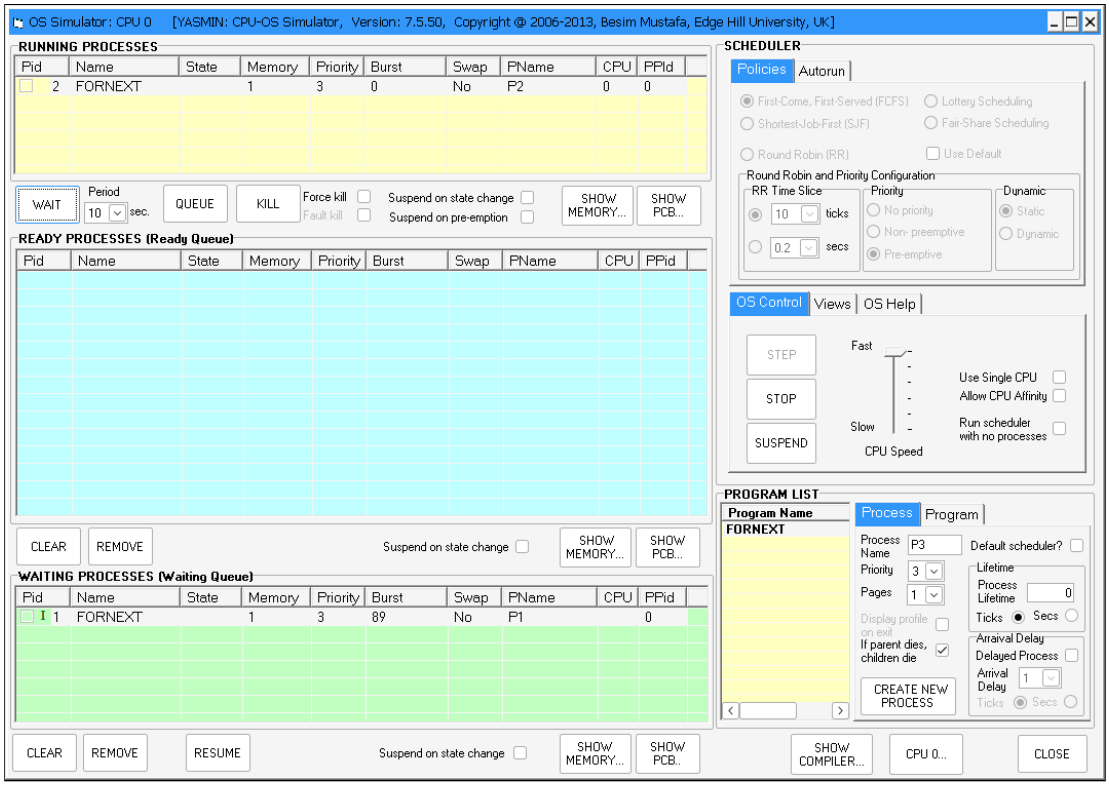


Figure 10. P1 in the Waiting queue (Burst 89) while P2 runs.

```
[OS0: 00000000]: P1(1)  moved from READY to RUNNING state on CPU 0
[OS0: 00000089]: P1(1)  moved from RUNNING to WAITING state
[OS0: 00000089]: P2(2)  moved from READY to RUNNING state on CPU 0
[OS0: 00000454]: P2(2)  terminated normally
[OS0: 00000455]: P1(1)  moved from WAITING to READY state
[OS0: 00000455]: P1(1)  moved from READY to RUNNING state on CPU 0
[OS0: 00000731]: P1(1)  terminated normally
```

Explain the behaviour of this process.

Pressing WAIT (period 10 s) moved P1 from the CPU to the waiting queue at tick 89, and P2 was given the CPU. P1 stayed waiting for the full 10 seconds even after P2 finished, so the CPU was idle for a while. Then P1 went back to the ready queue, got the CPU again and continued from where it stopped (not from the start), finishing at tick 731. Its tick count is still 365 (89 + 276) and its average burst is 182 because it ran in two bursts.

What state transitions did it go through?

New → Ready → Running → Waiting → Ready → Running → Terminated.

4.6 Viewing the Log (step 36)

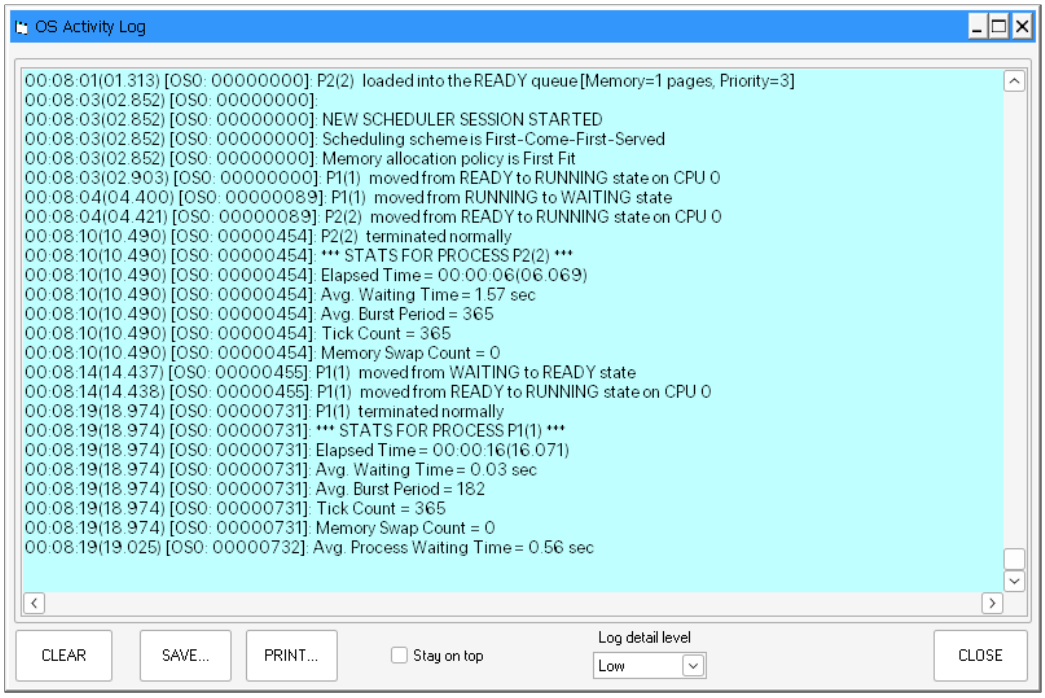


Figure 11. Log of the WAIT run.

Run	What the log shows (ticks)	Avg. waiting time
FCFS	P1 0–365, then P2 366–731	P1 0.05 s, P2 5.45 s
Priority NP	P2 0–365; P1/P4 alternate to 1091/1097; P3 1098–1463	0.03 / 0.44 / 0.45 / 23.42 s
Round Robin	P1, P2, P3 rotate every 10 ticks; end at 1085, 1091, 1097	0.49 / 0.5 / 0.51 s
Priority PE	P1 pre-empted at 4; P3 4–369; P1 370–731; P2 732–1097	P3 0.04, P1 0.64, P2 41.51 s
WAIT	P1 waiting at 89; P2 89–454; P1 back at 455, ends 731	P2 1.57, P1 0.03 s

The log confirms every action: the order of dispatch, each pre-emption, the move to the waiting state and the end of each process.

5. Optional: Killing a Process

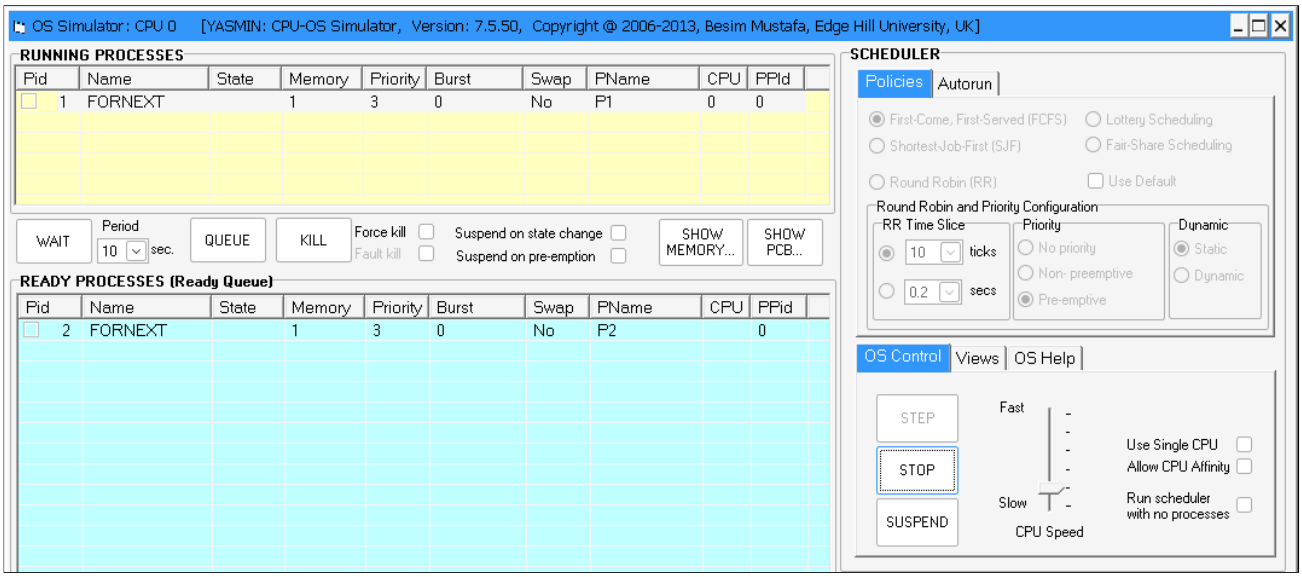


Figure 12. Before KILL: P1 running, P2 ready.

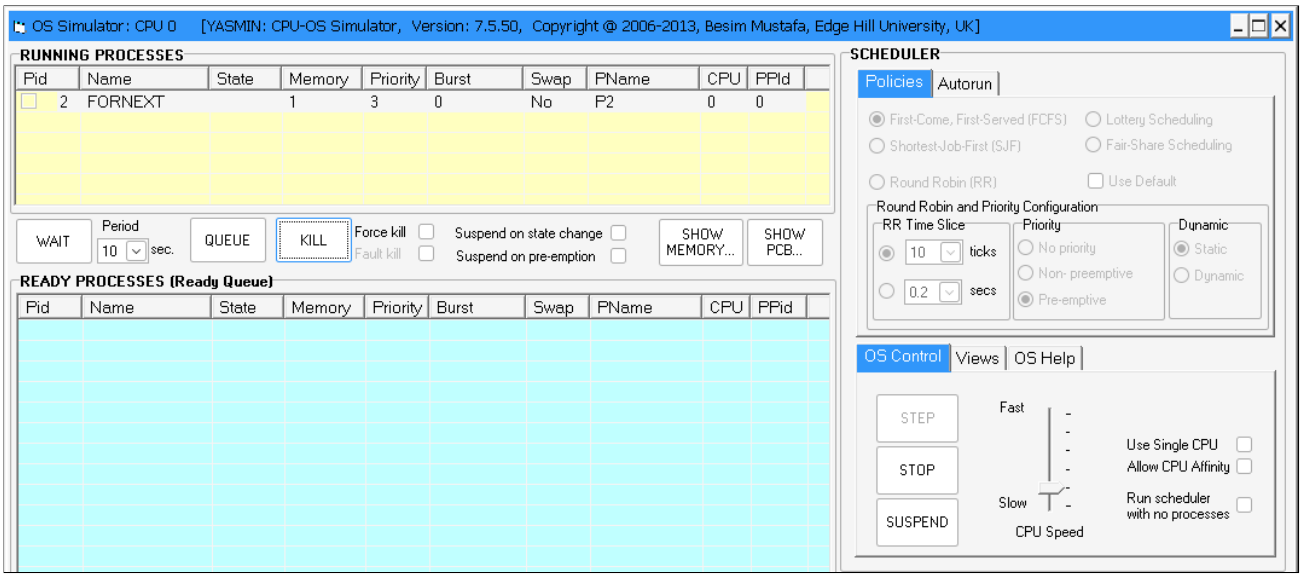


Figure 13. After KILL: P2 running, ready queue empty.

```
[OS0: 00000002]: P1(1)  killed
[OS0: 00000002]: P2(2)  moved from READY to RUNNING state on CPU 0
[OS0: 00000367]: P2(2)  terminated normally
```

What happened?

P1 was terminated immediately at tick 2 (Tick Count = 2) and removed from the system. It did not go back to any queue.

Is another process running?

Yes. The scheduler gave the CPU to P2 right away, and P2 ran normally to the end (tick 367).

6. Optional: Threads (THREADTEST)

The FORNEXT program was removed with REMOVE (confirmation “Do you really wish to remove the selected program?” → Yes). THREADTEST was loaded, one process was created and run (RR, 10 ticks), then suspended and viewed.

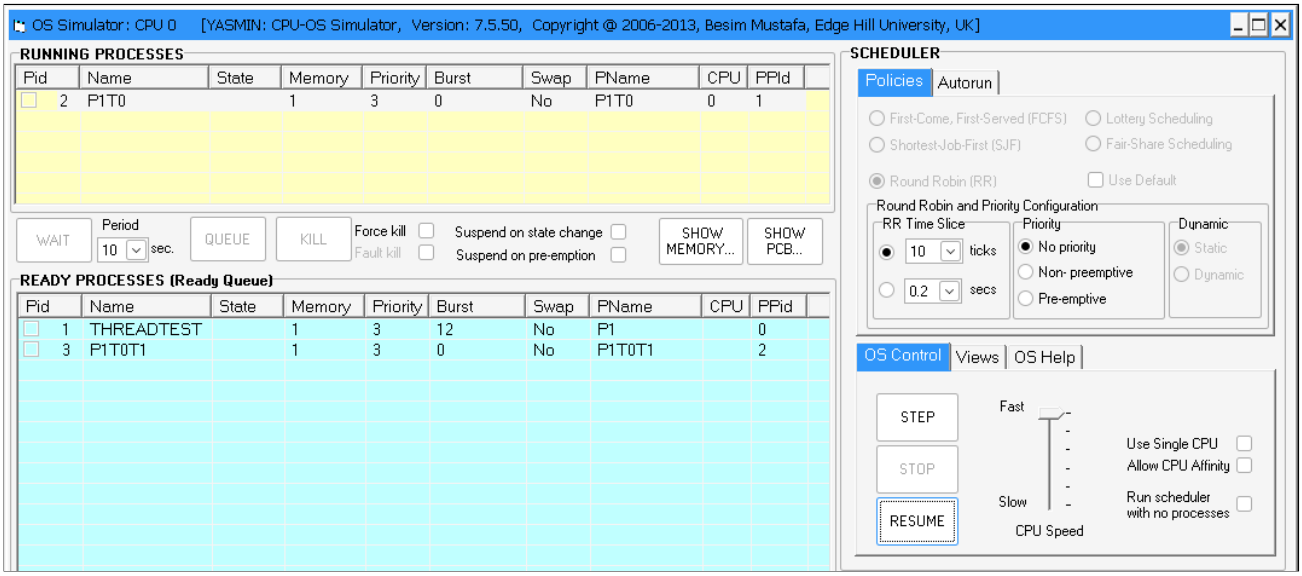


Figure 14. THREADTEST: three processes (P1, P1T0, P1T0T1).

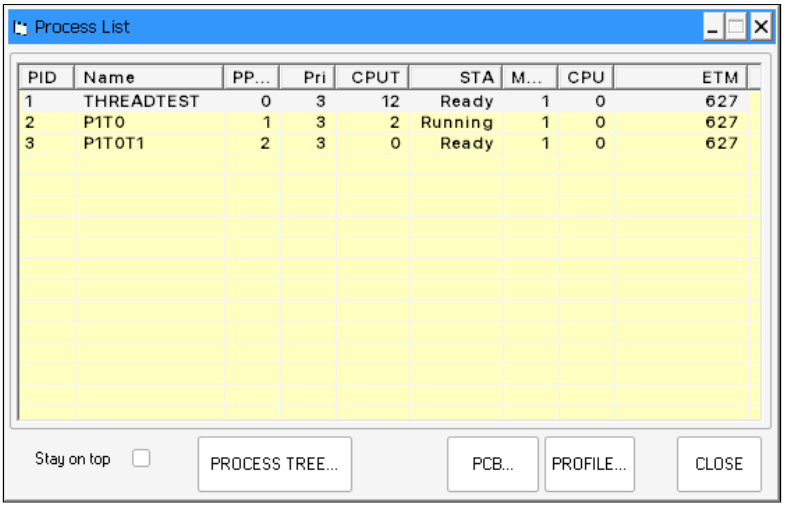


Figure 15. Process List.

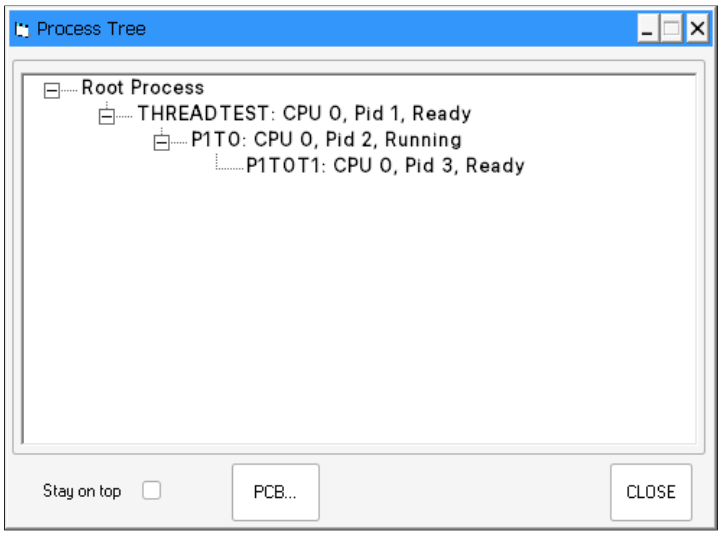


Figure 16. Process Tree.

What do you see? Can you explain what you see?

Three processes are shown although only one was created: THREADTEST (Pid 1, PPid 0), P1T0 (Pid 2, PPid 1) and P1T0T1 (Pid 3, PPid 2). The Process Tree shows Root Process → THREADTEST → P1T0 → P1T0T1, so each one is a child of the one before. The log shows P1T0 created at tick 2 and P1T0T1 at tick 14; all three then share the CPU in round robin. At the end the threads “stopped” (ticks 389 and 390), and the main process, which was waiting for them, terminated at tick 390.

Why are there three processes when you created only one?

Because the program creates threads. thread2 and thread1 are declared “as thread”, so calling them does not run them as normal subroutines; it asks the OS (SWI 5) to create a new thread. The main program created thread2 (P1T0) and thread2 created thread1 (P1T0T1). A thread has its own PC, registers and stack but runs code of the same program, and the simulator schedules threads like processes, so they appear as separate entries with their own PID and parent PID.

7. Conclusion

Policy	Pre-emptive?	Strength	Weakness
FCFS	No	Simple, few switches	Long waits behind long jobs
Priority (NP)	Only at slot end	Important jobs first	Starvation of low priority
Round Robin	Yes (time slot)	Fair, good response	Many context switches
Priority (PE)	Yes (on arrival)	Fastest response for urgent jobs	Starvation, more switches

The lab showed how the OS creates processes, keeps their state in the PCB, moves them between the ready, running and waiting states, and how the choice of scheduling policy changes the order and waiting time of the processes.

Appendix: Programs Used

The .sas files were not included in the download, so they were written in the simulator's compiler.

```
program FORNEXT
  var a integer
  for n = 1 to 40
    a = a + n
  next
end
```

```
program THREADTEST
  sub thread1 as thread
    writeln("In thread1")
    for i = 1 to 30
    next
  end sub

  sub thread2 as thread
    call thread1
    writeln("In thread2")
    for j = 1 to 30
    next
  end sub

  call thread2
  writeln("In main")
  for k = 1 to 30
  next
  wait
end
```

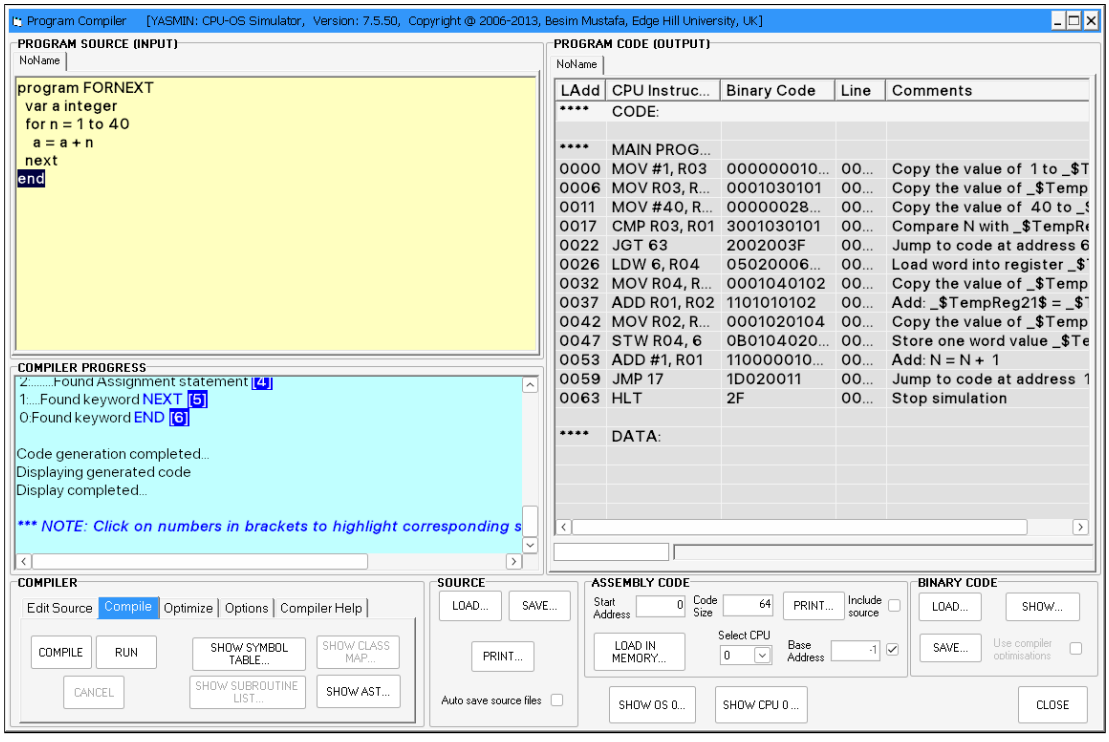


Figure 17. FORNEXT compiled code (address 17 = CMP R03,R01).